

LifeOS

Personal Life Operating System

Architecture & Design Document

Version 0.2 • March 2026

0. The Vision

Most of what we call work in daily life is not real thinking. It is coordination overhead. Researching where to invest, writing a grocery list, planning a trip, drafting a task plan for the week — none of this requires you specifically. It just requires context about you.

LifeOS is a personal AI operating system that runs continuously in the background. You give it a north star — what you want to achieve, who you want to become — and it handles the operational work across every domain of your life: finance, health, career, travel, logistics. You review, approve, and enjoy the output. The cognitive load of coordination disappears.

Core principle: You should spend your time thinking, deciding, and experiencing — not managing. LifeOS manages so you do not have to.

This is different from existing AI tools like ChatGPT or Perplexity, which are reactive — you ask, they answer. LifeOS is proactive. It knows your goals, watches your life passively, and surfaces work to you already done or ready to approve. The interaction model is closer to a chief of staff than a search engine.

1. Architecture Overview

LifeOS is built from six distinct layers that work together. Each layer has one clear job. Together they form a pipeline from raw perception of your world to real-world execution on your behalf.

- Perception Layer — the eyes and ears. Captures everything happening in your world.
- Context Layer — the memory. Stores a structured, persistent model of who you are, how you decide, and what you are working toward.
- Orchestrator — the brain. Translates your goals into tasks and routes them to the right agents.
- Agent Layer — the hands. Specialist agents that do domain-specific work, built on a plug-and-play model.
- Approval Layer — the checkpoint. Nothing consequential happens without your sign-off.
- Onboarding — the starting point. Populates the context layer on day one so the system is useful immediately.

Data flows in one direction: Perception feeds Context, Context informs the Orchestrator, the Orchestrator directs Agents, Agents surface output through the Approval Layer, and your approvals and overrides feed back into Context. The loop is complete and self-improving.

2. Perception Layer

The Perception Layer is how LifeOS understands your world without you having to manually update it. It runs continuously and passively, capturing two streams of information: what you hear and say, and what you see on screen.

2.1 Microphone Input

A microphone runs continuously in the background and captures all audio — your conversations, calls, podcasts, videos you watch, meetings you attend. This is transcribed locally using Whisper (an open-source speech-to-text model by OpenAI) and written to the ledger.

The microphone is the primary passive input channel. Most of what matters about your life passes through audio — decisions you make in conversations, preferences you express, commitments you give, things you learn while listening. Capturing this means the system learns about you without any deliberate effort on your part.

Why continuous? Manually logging your life is friction. The moment you have to remember to update a system, it becomes a task in itself. Running continuously removes this friction entirely. The system observes your life the way a brilliant assistant sitting next to you all day would.

2.2 Screen Input

Screen capture is a universal integration fallback. Many applications do not expose APIs, or their APIs are paywalled, or they change frequently. Rather than building and maintaining dozens of integrations, the screen reader simply sees what you see and extracts relevant context from it.

Screen capture does not need to run continuously like the microphone. It operates in three modes:

- On-demand: you explicitly ask the system to look at your screen
- Triggered: the system activates when you open a specific app that has no native integration
- Periodic: a lightweight screenshot every few minutes when the system is active

Why screen over integrations? Native integrations break when apps update their APIs, move features behind paywalls, or deprecate endpoints. The screen is a stable universal interface — it works with any app, forever, because it sees exactly what you see.

2.3 Native Integrations

Where apps do provide good APIs, native integrations are preferred because they give structured, reliable, low-cost data. Examples include Gmail, Google Calendar, GitHub, Notion, and Spotify. The priority rule is simple: prefer native integration where it exists, fall back to screen capture where it does not, and let the microphone fill in the rest.

2.4 Wake Word and Intent Detection

Not everything the microphone hears is a command to LifeOS. Most audio is ambient — conversations, background noise, media. The system must distinguish between you talking to someone else and you talking to LifeOS.

This works the same way as Siri or Alexa: a lightweight always-on wake word detector listens for a specific trigger phrase. When detected, the system shifts from passive capture mode to active command mode and routes your intent to the Orchestrator. Everything else stays in passive mode and flows only to the ledger and extraction pipeline.

3. The Ledger

The Ledger is a permanent, append-only record of everything the Perception Layer captures. It is never modified, never deleted — only written to. Think of it as a personal life log.

Every entry in the ledger is timestamped and tagged with its source. A typical entry looks like this:

```
[2026-03-31 09:14:22] [mic] [source: call]
```

```
I think I want to lean more toward ML infrastructure, pure backend is getting repetitive
```

```
[2026-03-31 10:45:01] [screen] [source: browser - moneycontrol.com]
```

```
User viewing HDFC Bank, price 1642, checking 52-week high
```

```
[2026-03-31 11:20:44] [mic] [source: ambient]
```

```
remind me to look into Anyscale pricing before the LinkedIn internship starts
```

The Ledger serves three purposes. First, it is the raw material for the extraction pipeline — the Context Layer is derived from it. Second, it is a searchable personal memory — you can query it directly to recall what you decided or discussed weeks ago. Third, it is a safety net — if the extraction layer misses something or misclassifies it, the Ledger preserves the original and allows re-processing.

Storage format: SQLite is the recommended storage format rather than a flat text file. It preserves the same append-only semantics but gives you free querying, indexing by timestamp and source, and efficient storage as the file grows over months and years.

4. Context Layer

The Context Layer is the most important component of LifeOS. It is a structured, persistent model of you — your goals, constraints, preferences, finances, health, schedule, judgement patterns, and north stars. Every agent reads from it. Without it, agents are just generic AI tools. With it, they become your personal specialists.

The context layer is the primary moat of LifeOS. Every day it runs, it knows you better. After six months, it contains something no competitor can replicate — a deep, accurate, continuously updated model of you specifically. The switching cost grows every day.

4.1 Extraction Pipeline

The Ledger is raw and noisy. The Context Layer is distilled and structured. A background extraction job runs periodically — every few minutes — reading new Ledger entries since the last run, passing them through an LLM, and writing only meaningful context deltas to the Context Layer.

The extraction layer asks one question about every Ledger entry: does this tell me something new, meaningful, and durable about this person? A preference expressed, a

decision made, a goal updated, a commitment given — these are context-worthy. Background noise, filler conversation, and routine observations are discarded.

4.2 The Five Documents

The Context Layer is made up of five living documents. Each has a single clear responsibility. Together they form a complete model of a person.

`context/`

```
public.json          <- who you are. freely read by all agents.

private.md           <- sensitive details. gated, permission-required.

privacy_rules.md     <- who can access what, and under what conditions.

judgement_rules.md   <- how you decide. confidence-scored, self-updating.

north_stars.md       <- what you are working toward, across all time horizons.
```

Public context

Freely available to all agents in every interaction. Contains your general goals, skill set, schedule preferences, dietary habits, interests, and risk appetite. This is what the system reads by default. Populated initially through onboarding, then updated continuously by the extraction pipeline.

Private context

Sensitive information that agents can request access to but do not read automatically. Includes specific account numbers and balances, medical details, relationship dynamics, and anything flagged as private. When an agent needs private context, it raises a request. You approve. That session uses it and closes cleanly without bleeding into regular history.

The incognito model: Think of private context access as an incognito session. The data is still present in the system — the same tradeoff you accept when chatting with Claude or OpenAI today. But it is compartmentalized: gated, session-scoped, and not freely mixed into regular context.

Privacy rules

A document you can edit directly that defines access control rules across the system. Which agents can request private context. Which agents have automatic access. Which topics always route to private context regardless of source. Adjustable at any time.

Judgement rules

A living document that writes itself entirely from watching you make decisions. You never touch it directly. Every approval, override, and answered question writes a delta to it. Over time it becomes a distilled model of how you think and decide — not just what you prefer, but your reasoning patterns.

A typical entry looks like this:

Finance:

- Willing to take higher risk on small cap tech [confidence: 8/10]
- Never approves anything over 50,000 without full breakdown [confidence: 10/10]
- Rejected HDFC Bank three times — do not suggest without new catalyst [confidence: 7/10]

Travel:

- Layover acceptable if savings exceed 10,000 rupees [confidence: 9/10]
- Always prefers window seat [confidence: 10/10]
- Never books non-refundable hotels more than 2 months out [confidence: 6/10]

Each rule carries a confidence score — how many times this pattern has been observed. A rule seen once is a hypothesis. A rule seen twenty times is a reliable preference. The Orchestrator weights rules by confidence when deciding whether to auto-approve or still surface a question. This prevents one unusual decision from permanently changing the system's behavior.

The judgement rules document is what compounds over time. The context layer stores what you know and prefer. The judgement rules store how you decide. Together they are not just a profile — they are a behavioral model. Recreating your context layer elsewhere is painful. Recreating six months of accumulated judgement patterns is nearly impossible.

North stars

A document containing your goals across every time horizon simultaneously. Every action the Orchestrator takes, every suggestion an agent surfaces — it first checks the north stars and asks: does this serve them? If an agent is about to suggest something that conflicts with a north star, it flags the tension rather than resolving it silently.

North stars are not limited to one window. You can have multiple simultaneously — a lifetime direction, a five-year ambition, a three-month project goal, a weekly focus. The system evaluates actions against this hierarchy from the bottom up: weekly first, then upward. When two north stars conflict — a weekend trip conflicts with a savings goal — the system surfaces that tension to you rather than picking a side.

A typical north stars document looks like this:

```
Lifetime:    Financial independence, meaningful technical work at scale
5-year:      Principal / Staff engineer at a top infrastructure company
1-year:      Crush LinkedIn internship, publish ML research paper
3-month:     Ship LifeOS v1, complete CS271 with strong grade
This week:   Finish architecture doc, start Phase 2 of hallucination project
```

Why multiple horizons? A single six-month goal is too rigid for real life. Goals exist at every scale simultaneously. The north stars model lets the system serve your weekly focus without losing sight of your lifetime direction. A suggestion that helps this week but hurts the five-year goal gets flagged. One that aligns across all levels gets prioritized.

4.3 Feedback Loop

The Context Layer is not static. Every time you approve, override, or answer a question, that response flows back as a delta into both public context and judgement rules. The system learns from your decisions continuously, without any manual input required. The longer it runs, the less it needs to ask, and the better its defaults become.

5. Onboarding — Solving the Cold Start

On day one, the context layer is empty. The judgement rules are blank. The north stars are undefined. A system that knows nothing about you cannot be useful, and a system

that is not immediately useful will not be used long enough to learn. This is the cold start problem.

Onboarding solves it by seeding the context layer before passive perception has had time to build it naturally. It happens in two phases.

5.1 Structured Questionnaire

When you first set up LifeOS, it asks you a short set of structured questions across each domain — your current goals, financial situation in broad strokes, dietary preferences, work setup, travel patterns, and anything else the agents need to be immediately useful. This is a deliberate 20-minute investment that pays off immediately.

The questionnaire also captures your north stars. You define your goals across time horizons — lifetime, five years, one year, this quarter, this week. This becomes the steering document for everything the Orchestrator does from day one.

5.2 Document Ingestion

Beyond the questionnaire, you can upload documents that describe your current life state. Your resume tells the system your professional background, skills, and career trajectory. SOPs and personal notes tell it how you like to work. Financial statements give it a snapshot of your current position. Health records tell it your baseline.

The extraction pipeline processes uploaded documents exactly the same way it processes ledger entries — pulling context deltas and writing them to the appropriate documents in the context layer. Someone who uploads ten documents on day one has a context layer that would otherwise take weeks of passive capture to build naturally.

Why document ingestion matters: Passive perception is powerful but slow. It takes weeks of continuous listening to learn things a resume tells you in seconds. Document

ingestion gives the system a running start and makes it genuinely useful from the first day rather than requiring a patience tax before it earns your trust.

6. The Orchestrator

The Orchestrator is the brain of LifeOS. It sits between the Context Layer and the Agent Layer. Its job is to take your goals and incoming intents, translate them into concrete coordinated work, and route that work to the right agents.

6.1 What It Does

When you state an intent — either via voice command or through a scheduled goal review — the Orchestrator does four things. First, it reads the relevant slice of your Context Layer to understand constraints, preferences, and prior decisions. Second, it checks your north stars to ensure the task serves your goals across time horizons. Third, it decomposes the intent into sub-tasks and identifies which agents are responsible for each. Fourth, it creates a Task Context Object and coordinates execution across all involved agents.

6.2 North Star Check

Before routing any task, the Orchestrator checks it against the north stars document. If the task aligns, it proceeds. If it conflicts with a goal at any time horizon, the Orchestrator flags the tension in the approval layer before any work is done. It never silently resolves a north star conflict — it always surfaces it to you.

For example: you ask the system to book a weekend trip to Goa. The Orchestrator checks north stars. Your three-month north star is shipping LifeOS v1. Your one-year north star includes a savings target. Before routing to the travel agent, it surfaces: this trip costs 18,000 rupees and falls during a week you had marked as a deep work block. Do you want to proceed? You decide.

6.3 Trigger Model

In the current version, the Orchestrator is triggered manually via voice command with a wake word. You state your intent, and it takes over. This is intentional — starting with manual triggers keeps the system predictable and prevents unwanted autonomous actions while trust is being established.

In future versions, the Orchestrator can be made proactive — waking itself on a schedule, or in response to signals from the Perception Layer. A Monday morning briefing that reviews progress toward your north stars and surfaces the week's agenda is the first natural step toward this.

6.4 Task Decomposition

The Orchestrator breaks complex intents into discrete, agent-sized sub-tasks. It identifies dependencies between sub-tasks — which must complete before another can begin — and which can run in parallel. It then routes each sub-task to the appropriate agent with the relevant slice of context attached.

7. Agent Layer

Agents are niche specialists. Each agent knows one domain deeply and operates only within it. They do not talk to each other directly. They communicate through the shared Task Context Object managed by the Orchestrator.

7.1 Plug-and-Play Agent Model

The agent layer is designed as a plugin registry. The Orchestrator does not need to know what agents exist in advance — it reads the registry at runtime and routes tasks to whatever is registered. Adding a new agent means dropping it into the registry with a standard interface. No changes to the Orchestrator, the Context Layer, or any other agent are required.

Every agent in the registry declares four things:

- Domain: what category of tasks it handles
- Context requirements: which slice of the context layer it needs to operate
- Tools: what external tools, APIs, or capabilities it has access to
- Task interface: what kinds of tasks it can accept and what output format it produces

This means the system is infinitely extensible. You want a legal agent that reviews contracts? Register it. A learning agent that builds study plans from your north stars? Register it. A relationships agent that tracks important people in your life and surfaces when you should reach out? Register it. The architecture supports it without modification.

It also means the underlying model powering any agent is swappable. If a better model is released tomorrow, you point the agent at the new model and you are done. The rest of the system is untouched.

Why plug-and-play? Your life will change. The agents you need in six months are not exactly the agents you need today. A rigid system requires surgery to extend. A plugin registry treats extensibility as a first-class design property — new capability is additive, not disruptive.

7.2 Initial Agent Set

The starting set of agents covers the highest-value domains for daily life:

- Finance Agent: stock research, portfolio review, budgeting, expense analysis, savings planning, forex for travel
- Health Agent: meal planning, grocery list generation, dietary tracking, supplement reminders, workout planning
- Work Agent: task planning, coding assistance, calendar management, meeting prep, project tracking
- Travel Agent: flight and hotel research, itinerary drafting, visa requirements, local recommendations
- Research Agent: general research on any topic, summarising articles and papers, competitive analysis

7.3 The Task Context Object

The Task Context Object is the shared workspace created by the Orchestrator for every multi-agent task. It replaces direct agent-to-agent communication. Agents read from it, write their outputs to it, and raise questions through it. They never need to know which other agents exist or what they are doing.

```
TaskContext {
```

```

task_id          -> unique identifier for this task

intent           -> the original user request

triggered_by     -> source and timestamp

status           -> in_progress / awaiting_input / awaiting_approval / complete

north_star_check -> alignment with active north stars, any conflicts flagged


user_context     -> relevant slice from the Context Layer


agents_involved  -> list of agents assigned to this task


shared_state     -> where agents read and write their outputs

{
    travel      -> { flights: [...], hotels: [...], itinerary: draft }

    finance     -> { estimated_cost: X, budget_ok: true }

    health      -> { dietary_notes: Z, vaccines: [...] }

    work        -> { conflicts: [], calendar_blocked: true }
}


dependencies     -> ordered constraints between agent tasks

questions        -> raised by agents when genuinely unsure

approval_items   -> decisions requiring your sign-off

final_output     -> assembled by Orchestrator from shared_state
}

```

7.4 The If-Unsure-Ask Rule

Agents follow a clear decision hierarchy when they encounter ambiguity:

- First: check the Context Layer and Judgement Rules. The answer is probably already there.
- Second: make a reasonable default, proceed, and flag it transparently in the approval layer so you can override it.
- Third: if the decision is genuinely consequential and no clear default exists — stop and ask.

When an agent asks, it writes a question into its slot in the `shared_state` and sets the task status to `awaiting_input`. The Orchestrator surfaces the question through the interface. You answer via voice. Your answer is written back into the Task Context Object, the agent resumes, and the answer is also added to judgement rules so the agent does not ask the same question twice.

Why this rule matters: Agents making assumptions silently is worse than agents asking. A wrong assumption compounds — the travel agent books the wrong hotel class because it assumed your budget was higher. The if-unsure-ask rule keeps you in control of consequential decisions while still automating everything that is clear. And every answer you give makes the system smarter.

8. Approval Layer

The Approval Layer is your primary interface with LifeOS. You do not interact with agents directly. You interact with a clean surface that shows you what the system has done, what it is about to do, and what it needs from you.

8.1 What It Shows

The Approval Layer surfaces three types of cards:

- Information cards: things the system has done autonomously and is reporting back (your weekly meal plan is ready, your portfolio summary for the week)
- Approval cards: consequential actions the system wants to take but needs your sign-off first (book this flight for 32,000 rupees, buy 5 shares of this stock)
- Question cards: genuine ambiguities an agent could not resolve from context or judgement rules alone

Before surfacing any card, the Orchestrator first checks the judgement rules. If a rule clearly covers the situation, it either auto-approves, auto-rejects, or pre-fills a recommendation. You only see a card if the situation is novel or the judgement rules are ambiguous. Over time the approval layer gets quieter — it asks you less and less as it learns your patterns.

8.2 Trust Calibration

Not all actions require the same level of oversight. Low-stakes, repeatable actions — generating a grocery list, summarising an article, drafting a meal plan — can be set to auto-approve. High-stakes, irreversible actions — moving money, booking non-refundable travel, making calendar commitments — always pause for explicit approval regardless of what the judgement rules say.

You can adjust the trust threshold for any action category at any time through the privacy rules document in the context layer.

8.3 Interface Design

The interface is intentionally minimal. It is not a chat app. It is a notification surface. Cards appear when the system has something to surface. You review, tap approve or override, and move on. The system does not demand your attention — it waits for yours.

Input to the system is always through the microphone. Output is always through the interface. You never type. You never open an app to start an interaction. The system comes to you when it has something worth your attention.

9. Communication Model

9.1 Input: Voice First

All interaction with LifeOS happens through voice. A wake word activates the Orchestrator. Everything else is passive ambient capture flowing into the ledger. There is no app to open, no text to type, no interface to navigate. The system integrates into your life without adding a new interaction overhead on top of it.

9.2 Output: Minimal Interface

The system surfaces results through a simple card-based interface available on your phone or desktop. Cards are informational, actionable, or question-based. The interface does not try to be a dashboard or a command center. It exists only to deliver things that need your attention and receive your response.

9.3 Proactive vs Reactive

In the current version the system is reactive — you trigger it with a voice command. In future versions it becomes proactive: it reviews your north stars each week, notices things in the ledger that warrant action, and surfaces work to you before you have to ask. The Monday morning briefing — a scheduled summary of progress toward your goals and the week ahead — is the first natural step toward this.

10. End-to-End Data Flow

Tracing a complete example — you say: plan a trip to Japan in June.

- Wake word detected. Orchestrator activates.
- Orchestrator reads Context Layer: your budget, dietary restrictions, leave balance, passport validity.
- Orchestrator checks North Stars: one-year goal includes a savings target. Three-month goal is shipping LifeOS v1. Flags potential tension — surfaces a note in the approval card at the end.
- Orchestrator creates a Task Context Object for the trip planning task.
- Work agent runs: checks calendar, identifies available dates in June, writes conflicts to `shared_state`.
- Finance agent runs in parallel: estimates trip cost, checks budget, checks savings target alignment, writes `budget_ok` and forex advice to `shared_state`.
- Travel agent runs after work and finance complete: searches flights and hotels within confirmed budget and dates. Checks judgement rules: window seat preferred, layover acceptable if savings exceed 10,000 rupees. Finds a qualifying option. Sets status to `awaiting_input` on one remaining question: hotel area preference not found in judgement rules.
- Approval Layer surfaces question card: do you want to stay in Shinjuku or Shibuya?
- You respond via voice: Shinjuku. Answer written to judgement rules for future reference.
- Travel agent finalises itinerary. Health agent runs: dietary notes for Japan, restaurant types to seek out, no vaccine requirements.

- Orchestrator assembles final output. Flags north star tension: trip costs 28,000 rupees and falls during a planned deep work week.
- Approval Layer surfaces approval card: complete trip plan with the north star conflict noted. You review, decide the trip is worth it, approve.
- Finance agent executes. Travel agent confirms booking. Judgement rules updated: Shinjuku preferred for Japan trips. Context Layer updated with trip details.

Total effort from you: one voice sentence, one question answered, one approval tapped. The north star tension was surfaced and you made an informed decision. Everything else was done by the system.

11. Open Questions

The following are deliberately left open for the next phase of design:

- Technical stack: which models to use for extraction, which for agents, local vs cloud for each component
- Ledger storage: growth management, rolling archives, compression strategy as months of data accumulate
- Proactive orchestration triggers: how the system decides on its own to do something without a voice command
- Multi-device sync: how the context layer stays consistent across phone, laptop, and future devices
- The first MVP: the smallest version of this that provides daily value and can be built and tested quickly

TODO: Error recovery: what happens when the system gets something wrong — the extraction layer misclassifies something, the judgement rules learn a bad pattern, or an agent acts on a wrong assumption. Need a mechanism for correcting the system's model without breaking accumulated good patterns. Think through rollback, manual override of specific rules, and confidence decay for stale judgement rules.

12. Closing Thought

LifeOS is not a productivity tool. It is not another app in your dock. It is an operating system for your life — one that runs silently, learns continuously, and acts on your behalf so that your attention goes only where it genuinely matters.

The architecture described here is deliberately simple at each layer. The Perception Layer captures. The Context Layer remembers and learns. The Orchestrator coordinates with your goals always in view. Agents specialise and are swappable. The Approval Layer decides what needs you and what does not. Each layer has one job. Together they are more capable than any one of them alone.

The moat is not the technology. The moat is the context — six months of accumulated knowledge about how you think, what you want, and how you decide. A competitor can copy the architecture in a weekend. They cannot copy what the system learns about you over time. That asymmetry grows every day you use it.

North star: You set the destination. LifeOS handles the navigation. You live the journey.

Update log

Updated Ingestion & Memory Pipeline

The ingestion pipeline is the entry point of the system and is responsible for continuously capturing raw data from the Perception Layer, including microphone input, screen activity, and native integrations. This data is written to the Ledger as short-term memory in an append-only format. The Ledger stores a complete but noisy record of recent activity for a limited time window (e.g., 3–4 days). It acts as a temporary buffer that preserves full fidelity of events while avoiding long-term storage of redundant or low-signal information.

A background processing layer then operates on this short-term memory. First, an LLM summarizes the raw data into **daily logs**, which form a mid-term memory layer. These summaries are clean, structured, and human-readable, capturing the key events, decisions, and activities of each day. In parallel, an extraction step identifies meaningful, durable insights from the same data and updates the Context Layer. This ensures that only important patterns—such as preferences, goals, constraints, and decision behaviors—are promoted to long-term memory, while transient details remain in summaries.

The Context Layer therefore contains two complementary components: **daily summarized logs** and **structured context files**. The summaries provide recent situational awareness (“what happened”), while the context files represent an evolving model of the user (“what we know about the user”). This separation allows the system to operate across time horizons—using recent summaries for short-term relevance and structured context for long-term personalization—while ensuring that memory remains both scalable and meaningful over time.

Design Principle — Incremental Knowledge Compilation

The LifeOS Context Layer builds on the same core idea as an LLM Wiki: instead of repeatedly searching raw data, the system continuously compiles information into a persistent, structured layer that improves over time. In traditional systems, knowledge is reconstructed from raw inputs on every query. In LifeOS, raw inputs from the Perception Layer are first stored in the Ledger, then distilled through the ingestion and extraction pipeline into durable context such as goals, preferences, constraints, and judgement rules.

This allows the system to operate on an already-organized understanding rather than re-deriving knowledge for every task. While an LLM Wiki organizes information about topics, LifeOS extends this mechanism to build a structured, evolving model of the user. The result is a system that compounds understanding over time and enables more accurate, personalized, and proactive decision-making.

Interface & Deployment Model — CLI + Local Dashboard

LifeOS is designed as a **local-first system** with a dual interface: a CLI as the primary runtime engine and a lightweight local web dashboard for configuration and control. The CLI handles core operations such as ingestion, summarization, context updates, querying, and task execution, enabling direct interaction with the system and seamless access to local files and data. The dashboard complements this by providing a user-friendly interface for onboarding, managing integrations, editing context and rules, reviewing daily summaries, and approving sensitive actions. All components run locally on the user's machine, with intelligence powered by the user's chosen LLM/API key, ensuring privacy, flexibility, and full user control over data and system behavior.

Existing Landscape and Positioning of LifeOS

Several tools today explore parts of the LifeOS vision, but none combine all its components into a single coherent system. AI assistants with memory (like ChatGPT or Rewind) capture and recall information but lack structured decision-making and action orchestration. Automation tools and agent frameworks focus on executing tasks but have weak long-term memory and little personalization. Knowledge systems like [Obsidian](#) or [LLM Wiki](#) excel at organizing information but do not act or make decisions. A few emerging “AI assistant” products attempt passive data capture and contextual help, but they still fall short in building a deep, structured model of the user or reliably driving actions.

LifeOS sits at the intersection of all these categories by combining continuous perception, structured personal memory, decision modeling, agent-based execution, and an approval loop into a unified system. While existing tools solve isolated layers, LifeOS aims to integrate them into a single pipeline that compounds understanding over time and translates it into meaningful,

personalized actions. This gap—between knowing information and acting intelligently on it—is where LifeOS differentiates itself.